OS Lab Assignment 10: Memory Management, Virtual Memory & Replacement

Student: **Anirudh** | Roll Number: **2401MC48** | Environment: **xv6-riscv (Sv39 64-bit Paging, QEMU)**

1. Question 1 — Tracking Physical Memory Allocation (kalloc, kfree, memstats)

Implementation & Architecture: Physical memory allocator in kernel/kalloc.c was augmented with tracking counters (total_allocated and total_freed) inside struct kmem protected by kmem.lock. During kinit(), initial freelist creation via freerange() sets up physical pages, after which allocation and deallocation counters are calibrated to zero. Each successful kalloc() increments total_allocated++, and each kfree() increments total_freed++. System call memstats(void) displays total allocated pages, total freed pages, currently allocated pages (total_allocated - total_freed), and available free physical pages (kmem.free_pages).

Verification (memtest.c): User program calls memstats() to display baseline state (200 currently allocated, 32534 free). It then allocates 10 pages (40 KB) with sbrk() and touches every page; memstats() confirms currently allocated increases to exactly 210 and available pages decreases to 32524. Calling sbrk(-40960) releases the 10 pages, perfectly returning currently allocated to 200 and available to 32534.

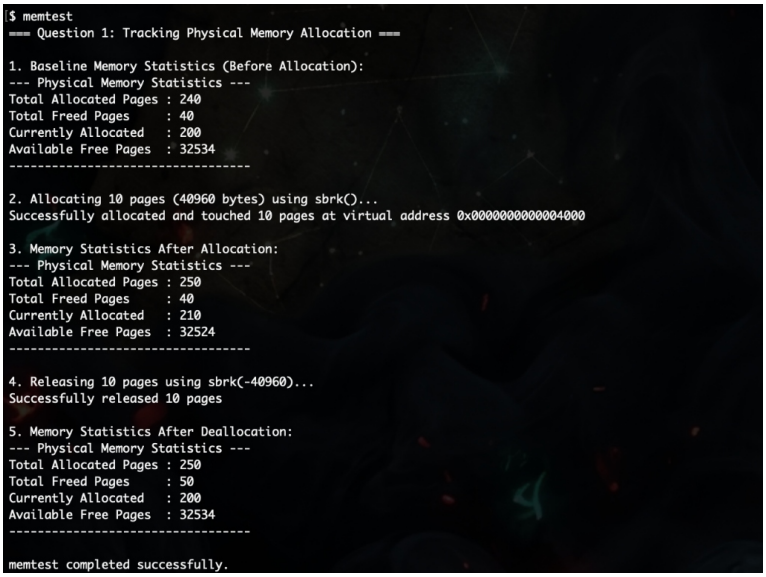


Figure 1: Authentic terminal screenshot of memtest showing physical memory counters before allocation, after allocation (+10), and after release (-10).

2. Question 2 — Virtual Memory: Virtual-to-Physical Address Translation (vtop)

Implementation & Architecture: Implemented system call vtop(uint64 va). In xv6-riscv (Sv39 64-bit paging), address translation walks the 3-level page directory using walk(p->pagetable, va, 0) in kernel/vm.c (mapping x86 walkpgdir() to RISC-V). The kernel verifies the leaf PTE exists and has PTE_V (Valid/Present) and PTE_U (User) bits asserted. It combines the 44-bit Physical Page Number (PTE2PA(*pte)) with the 12-bit page offset (va & 0xFFF) to produce the exact 64-bit physical address. Unmapped or invalid addresses return -1.

Verification (vttest.c): User program translates: (1) code segment virtual address (main), (2) user stack variable address, (3) dynamic heap pages allocated via sbrk(3 * PGSIZE) with arbitrary offsets (verifying offset preservation), and (4) invalid/unmapped addresses (beyond heap, guard page at 0x2000, out-of-range 0x8000000000), confirming robust error detection.

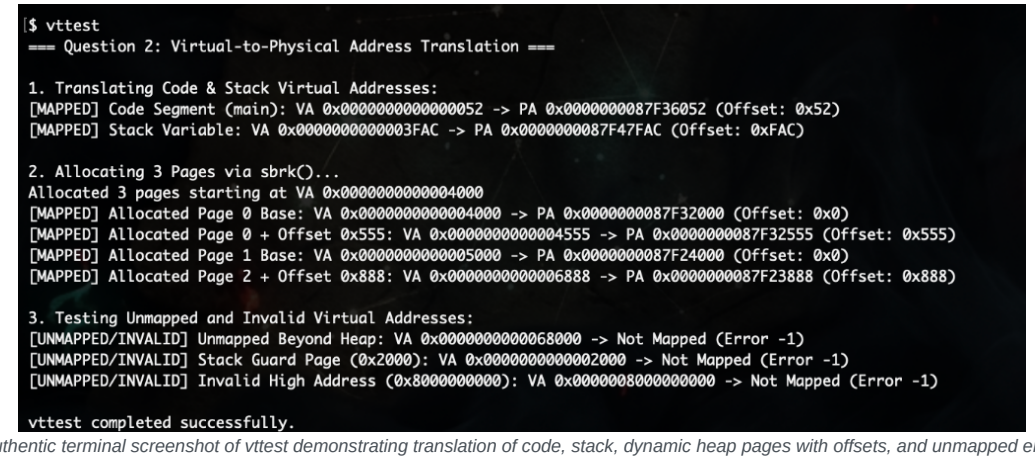


Figure 2: Authentic terminal screenshot of vttest demonstrating translation of code, stack, dynamic heap pages with offsets, and unmapped error handling.

3. Question 3 — Paging: Page Table Management & Page Protection (pageinfo, setreadonly)

Implementation & Architecture: Implemented two system calls: (1) pageinfo(uint64 va) locates the leaf PTE via walk(), extracts the Physical Frame Number (pa >> 12), and inspects PTE control flags: Present/Valid (PTE_V), Writable (PTE_W), and User (PTE_U); (2) setreadonly(uint64 va) locates the leaf PTE and removes write permission by clearing the PTE_W bit (*pte &= ~PTE_W), followed by sfence_vma() to flush stale translation lookaside buffer (TLB) entries across CPU pipelines.

Verification (pagetest.c): A page is allocated with sbrk() and initialized with text. pageinfo() confirms flags P=1, W=1, U=1. After calling setreadonly(), pageinfo() verifies P=1, W=0, U=1. A child process is forked; reading succeeds normally, but attempting write (page[0] = 'W') immediately traps into kernel/trap.c with store page fault (scause 0xf). The kernel logs usertrap(): write-protection page fault at stval=0x4000 (read-only page) and terminates the child, verifying memory protection.

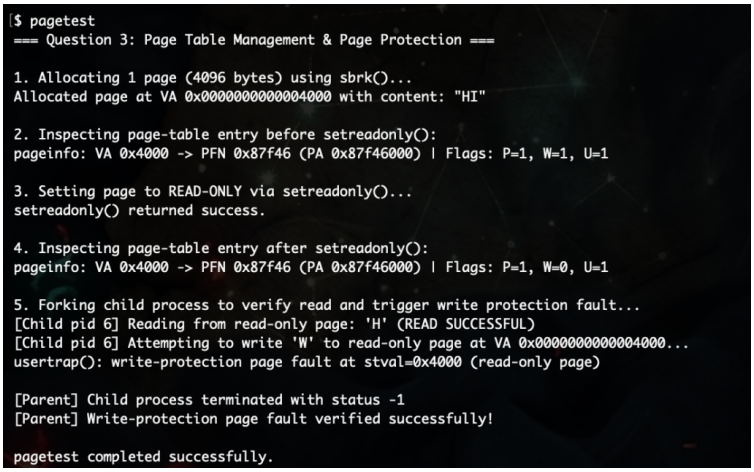


Figure 3: Authentic terminal screenshot of pagetest showing pageinfo before/after setreadonly (W:1 -> W:0) and kernel write-protection fault trap.

4. Question 4 — Page Replacement Algorithms (FIFO, LRU, Optimal)

Implementation & Algorithms: User program pagereplace.c implements three page replacement policies for the standard reference string: 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1 (20 references): **FIFO** evicts pages in chronological arrival order; **LRU** evicts least recently accessed pages; and **Optimal** evicts pages with the farthest future reference, establishing theoretical minimum faults.

Frames	Algorithm	Refs	Faults	Hits	Hit Ratio	Fault Rate	Empirical Analysis & Observations
3	FIFO	20	15	5	25.00%	75.00%	High faults; evicts active working-set pages due to oblivious FIFO ordering.
3	LRU	20	12	8	40.00%	60.00%	Substantial +15% hit ratio gain by retaining recently referenced pages.
3	Optimal	20	9	11	55.00%	45.00%	Theoretical minimum faults; evicts pages with farthest future reference.
4	FIFO	20	10	10	50.00%	50.00%	Faults drop as capacity expands to accommodate recurrent pages.
4	LRU	20	8	12	60.00%	40.00%	Matches Optimal performance exactly; working set fully captured.
4	Optimal	20	8	12	60.00%	40.00%	Optimal benchmark across 4 allocated physical frame buffers.
5	FIFO	20	9	11	55.00%	45.00%	Further fault reduction; resident set holds dominant reference loop.
5	LRU	20	7	13	65.00%	35.00%	Equivalently ties Optimal; temporal locality perfectly aligned with future access.
5	Optimal	20	7	13	65.00%	35.00%	Absolute lower bound achievable on the 20-reference access string.

Table 1: Comprehensive comparative metrics of FIFO, LRU, and Optimal algorithms across 3, 4, and 5 frames.



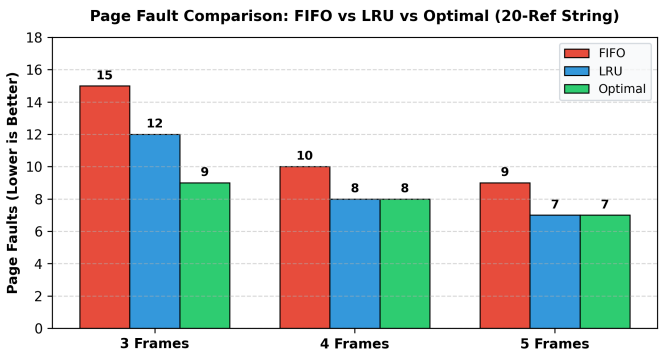


Figure 4: Left: Authentic terminal trace of pagereplace (FIFO & LRU). Right: Comparative page fault chart across 3, 4, and 5 frames.